

ProductDAOImpl.java

```
package com.harisudhan.harisudhanmart.dao;

import com.harisudhan.harisudhanmart.model.Product;
import java.math.BigDecimal;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.sql.Timestamp;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;
import javax.sql.DataSource;

public class ProductDAOImpl implements ProductDAO {

    private final DataSource dataSource;

    public ProductDAOImpl(DataSource dataSource) {
        this.dataSource = dataSource;
    }

    @Override
    public Product create(Product p) throws SQLException {
        String sql = "INSERT INTO products (seller_id, name, description, price, stock_qty, category, image_url) "
            + "VALUES (?, ?, ?, ?, ?, ?, ?)";
        try (Connection conn = dataSource.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
            ps.setLong(1, p.getSellerId());
            ps.setString(2, p.getName());
            ps.setString(3, p.getDescription());
            ps.setBigDecimal(4, p.getPrice());
            ps.setInt(5, p.getStockQty());
            ps.setString(6, p.getCategory());
            ps.setString(7, p.getImageUrl());
            ps.executeUpdate();
            try (ResultSet keys = ps.getGeneratedKeys()) {
                if (keys.next()) {
                    p.setId(keys.getLong(1));
                }
            }
            return p;
        }
    }

    @Override
    public Optional<Product> findById(long id) throws SQLException {
        String sql = "SELECT * FROM products WHERE id = ? AND active = TRUE";
        try (Connection conn = dataSource.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setLong(1, id);
            try (ResultSet rs = ps.executeQuery()) {
                return rs.next() ? Optional.of(map(rs)) : Optional.empty();
            }
        }
    }

    @Override
    public List<Product> search(String keyword, String category) throws SQLException {
        StringBuilder sql = new StringBuilder("SELECT * FROM products WHERE active = TRUE");
        List<String> params = new ArrayList<>();
        if (keyword != null && !keyword.isBlank()) {
```

```

        sql.append(" AND LOWER(name) LIKE ?");
        params.add("%" + keyword.toLowerCase() + "%");
    }
    if (category != null && !category.isBlank()) {
        sql.append(" AND category = ?");
        params.add(category);
    }
    sql.append(" ORDER BY created_at DESC");

    try (Connection conn = dataSource.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql.toString())) {
        for (int i = 0; i < params.size(); i++) {
            ps.setString(i + 1, params.get(i));
        }
        try (ResultSet rs = ps.executeQuery()) {
            List<Product> results = new ArrayList<>();
            while (rs.next()) {
                results.add(map(rs));
            }
            return results;
        }
    }
}

@Override
public List<Product> findBySeller(long sellerId) throws SQLException {
    String sql = "SELECT * FROM products WHERE seller_id = ? ORDER BY created_at DESC";
    try (Connection conn = dataSource.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setLong(1, sellerId);
        try (ResultSet rs = ps.executeQuery()) {
            List<Product> results = new ArrayList<>();
            while (rs.next()) {
                results.add(map(rs));
            }
            return results;
        }
    }
}

@Override
public boolean update(Product p) throws SQLException {
    String sql = "UPDATE products SET name = ?, description = ?, price = ?, stock_qty = ?, "
        + "category = ?, image_url = ? WHERE id = ? AND seller_id = ?";
    try (Connection conn = dataSource.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, p.getName());
        ps.setString(2, p.getDescription());
        ps.setBigDecimal(3, p.getPrice());
        ps.setInt(4, p.getStockQty());
        ps.setString(5, p.getCategory());
        ps.setString(6, p.getImageUrl());
        ps.setLong(7, p.getId());
        ps.setLong(8, p.getSellerId());
        return ps.executeUpdate() > 0;
    }
}

@Override
public boolean delete(long id, long sellerId) throws SQLException {
    String sql = "UPDATE products SET active = FALSE WHERE id = ? AND seller_id = ?";
    try (Connection conn = dataSource.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setLong(1, id);
        ps.setLong(2, sellerId);
        return ps.executeUpdate() > 0;
    }
}

```

```

    }
}

@Override
public boolean adminDelete(long id) throws SQLException {
    String sql = "UPDATE products SET active = FALSE WHERE id = ?";
    try (Connection conn = dataSource.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setLong(1, id);
        return ps.executeUpdate() > 0;
    }
}

private Product map(ResultSet rs) throws SQLException {
    Product p = new Product();
    p.setId(rs.getLong("id"));
    p.setSellerId(rs.getLong("seller_id"));
    p.setName(rs.getString("name"));
    p.setDescription(rs.getString("description"));
    p.setPrice(rs.getBigDecimal("price"));
    p.setStockQty(rs.getInt("stock_qty"));
    p.setCategory(rs.getString("category"));
    p.setImageUrl(rs.getString("image_url"));
    p.setActive(rs.getBoolean("active"));
    Timestamp ts = rs.getTimestamp("created_at");
    if (ts != null) {
        p.setCreatedAt(ts.toLocalDateTime());
    }
    return p;
}
}

```